Polish-Japanese Academy of Information Technology

Bachelor's engineering thesis

Short-Term Forecasting of Martian Surface Weather Using Machine Learning

PRO3, Task 05: Chapter 5, IT system architecture

Uladzislau Staliarou (s29905)

October 2026

5 IT system architecture

The results of this thesis are produced by a software system that downloads the archives of Chapter 3, builds the tables of Chapter 4, trains and scores the models of Chapter 6 and writes the numbers reported in Chapter 7. This chapter describes that system as an information system: its requirements and the architectural style chosen for them (Section 5.1), the hardware and runtime environments (Section 5.2), the software (Section 5.3), the data (Section 5.4), the integration of its parts and its use of the network (Section 5.5), security and integrity (Section 5.6), observability and testing (Section 5.7), its deployment, including the outline of a forecasting service announced in Chapter 1 (Section 5.8), and an assessment against common design considerations (Section 5.9). The models themselves are described in Chapter 6.

5.1 Requirements and architectural style

The system serves one purpose: to answer the three research questions with numbers that can be checked and repeated. Six requirements follow from the objectives of Chapter 1 and from the data of Chapters 3 and 4.

R1 *Traceability.* Every reported number must be traceable to the configuration, the code and the data files that produced it, and must be reproducible from them.

R2 *Comparability.* All models of an experiment must be scored on identical forecast issue times, targets and periods, because the research questions compare models, feature sets and sites with one another.

R3 *Leakage control.* No statistic of the validation or test periods may enter training (Chapter 4), and the test period may be used once per model.

R4 *Variation by configuration.* The model class, the feature set, the horizons and the site must be changeable without editing code, because each research question varies one of them while holding the others fixed.

R5 *Resource fit.* The system must run on a consumer laptop with 16 GB of memory and a 4 GB graphics processing unit (GPU), against about 150 GB of archive files on a local disk.

R6 *Single operator.* The system is built, run and maintained by one person, with no shared infrastructure.

These requirements led to a *layered, configuration-driven batch pipeline*: a set of command-line programs that run on one machine, call one shared Python package and pass data to one another only through files. In the vocabulary of software architecture, the code follows the layered pattern, in which each layer uses only the services of the layer below it, and the processing follows

the pipe-and-filter pattern, in which independent steps transform data streams in sequence[1]. In the classification of system types used in information technology, the system is a modular monolith: one code base, deployed as one unit, without services, message queues or a database server. Its stages correspond to the stages that industrial machine-learning workflows share, from data collection and cleaning through feature engineering, training and evaluation to deployment and monitoring[2], of which the last two exist here only as the outline of Section 5.8.

Three alternatives were considered and rejected. A service-oriented or event-driven design would suit concurrent users or streaming data, but the system has one user and no stream: the archives are published in periodic releases, months after the measurements (Chapter 3). Operating services would have added failure modes without adding capability. An experiment-tracking platform, such as MLflow[3], would record parameters, code versions and results in a server and a database; the plain-text run record described in Section 5.7 gives the same linkage between configuration, code and result, needs no running service, and remains readable without any tool. A database for the derived tables would suit concurrent updates and selective queries, but the tables are append-only time series that every program reads whole, an access pattern that column-oriented files serve directly. Finally, the interactive notebooks that often grow around such projects were excluded from the reproducible path: the notebook folder of the project is empty, and every step from download to result table is a program that can be run from the command line, as recommended for reproducible computational research[4]. The design also guards against two sources of technical debt known from machine-learning systems, the "glue code" that accumulates around libraries and the "pipeline jungles" of ad hoc data preparation steps[5]: all data preparation lives in one package with one order of steps (Chapter 4), and every processing program starts through one shared bootstrap.

5.2 Hardware and runtime environment

The system ran on two personal computers (Table 5.1). All experiments of the first project phase, and the final tuning and training of the temporal convolutional network (TCN), ran on a Windows laptop with a 4 GB NVIDIA GPU. Since October 2026 the project has lived on a MacBook with an Apple M4 processor, on which the data tables were rebuilt and all other final experiments

[1] Cf. L. Bass, P. Clements, and R. Kazman. *Software Architecture in Practice*. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2012.

[2] Cf. S. Amershi et al. "Software Engineering for Machine Learning: A Case Study". In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 291–300.

[3] M. Zaharia et al. "Accelerating the Machine Learning Lifecycle with MLflow". In: *IEEE Data Engineering Bulletin* 41.4 (2018), pp. 39–45.

[4] Cf. G. K. Sandve et al. "Ten Simple Rules for Reproducible Computational Research". In: *PLoS Computational Biology* 9.10, e1003285 (2013); G. Wilson et al. "Good enough practices in scientific computing". In: *PLoS Computational Biology* 13.6, e1005510 (2017).

[5] Cf. D. Sculley et al. "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems*. Vol. 28. Curran Associates, Inc., 2015. URL: `https://papers.nips.cc/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html` (accessed 2026-10-28).

were made. Both machines have 16 GB of memory, which bounds the design in two places. The largest objects held in memory are the feature table with secondary signals, 478,890 rows by 148 columns, and the training tensors of the TCN, which hold up to 121,795 windows of 288 steps by 14 channels in single precision, about 2 GB; both were sized to stay in memory with room to spare. The 4 GB of GPU memory dictated the batch size of 128, mixed-precision arithmetic and batched validation; the size of the network, 222,529 parameters, was chosen by validation error (Chapter 6). The source files, in contrast, never need to be in memory at once: the ingest programs read one sol of one sensor at a time, and the hourly tables of the two transfer sites are built by eight worker processes, each reading one file at a time.

Table 5.1: Hardware and runtime environments of the two machines

Property	Windows laptop	MacBook
Processor, memory	Intel x86-64, 16 GB	Apple M4 (10 cores), 16 GB
Graphics processor	NVIDIA GeForce RTX 3050 Ti Laptop, 4 GB, CUDA 12.1	Apple M4 GPU, shared memory, Metal Performance Shaders (MPS)
Operating system	Windows 10 (build 19045)	macOS 26.6
Python	3.11.5	3.12.14
Deep-learning build	PyTorch 2.2.2 with CUDA, mixed precision	PyTorch 2.2.2 for arm64, single precision
Other packages	data and model packages at identical pinned versions (NumPy 1.26.4, pandas 2.2.2, PyArrow 15.0.2, LightGBM 4.3.0, PyYAML 6.0.1), in a pip virtual environment with the package installed in editable mode; Matplotlib 3.8.4 and pytest 8.2.0 on the MacBook only	
Storage	local disk; archives, tables and run records	local disk; archives (151 GB), tables (0.4 GB), run records (0.2 GB)
Used for	all experiments until October 2026; six of the seven MEDA per-sensor caches; final TCN tuning and run (27 October 2026)	rebuilt 5-minute, hourly and feature tables; reference forecasts, GBDT, RQ2, RQ3 and all analyses (October 2026)

Source: own elaboration.

The runtime is Python, with the package versions pinned in an environment file for the conda package manager and in a requirements file for pip. The pins keep the two machines comparable: on the MacBook, the reference forecasts and the gradient-boosted decision trees (GBDT) reproduced the metrics of the Windows runs exactly. The one unavoidable difference is the GPU. CUDA, mixed precision and pinned host memory are specific to NVIDIA hardware, so the TCN code selects the device at run time, CUDA first, then the Apple GPU, then the processor, and enables mixed precision only on CUDA. On the Apple GPU the twelve TCN models trained in about 6.5 hours in single precision, against 31 minutes on the laptop, which is why the final TCN run was made there (Section 5.8). Google Colab was documented as a fallback for memory-bound runs and was never needed. One remark on hygiene is due: the

environment files also pin seven packages that the code never imports (XGBoost, scikit-learn, SciPy, statsmodels, seaborn, lxml and joblib); the packages actually used are those of Table 5.1 together with tqdm and tabulate.

5.3 Software architecture

Figure 5.1 shows the layers of the software and the files it reads and writes. The code comprises 4,622 lines of Python: 1,737 in the package, 2,321 in the command-line programs, the shared bootstrap and one coursework script, and 564 in the tests. It is organized by one rule: the programs call the package, and the package never calls the programs or reads the configuration files itself.

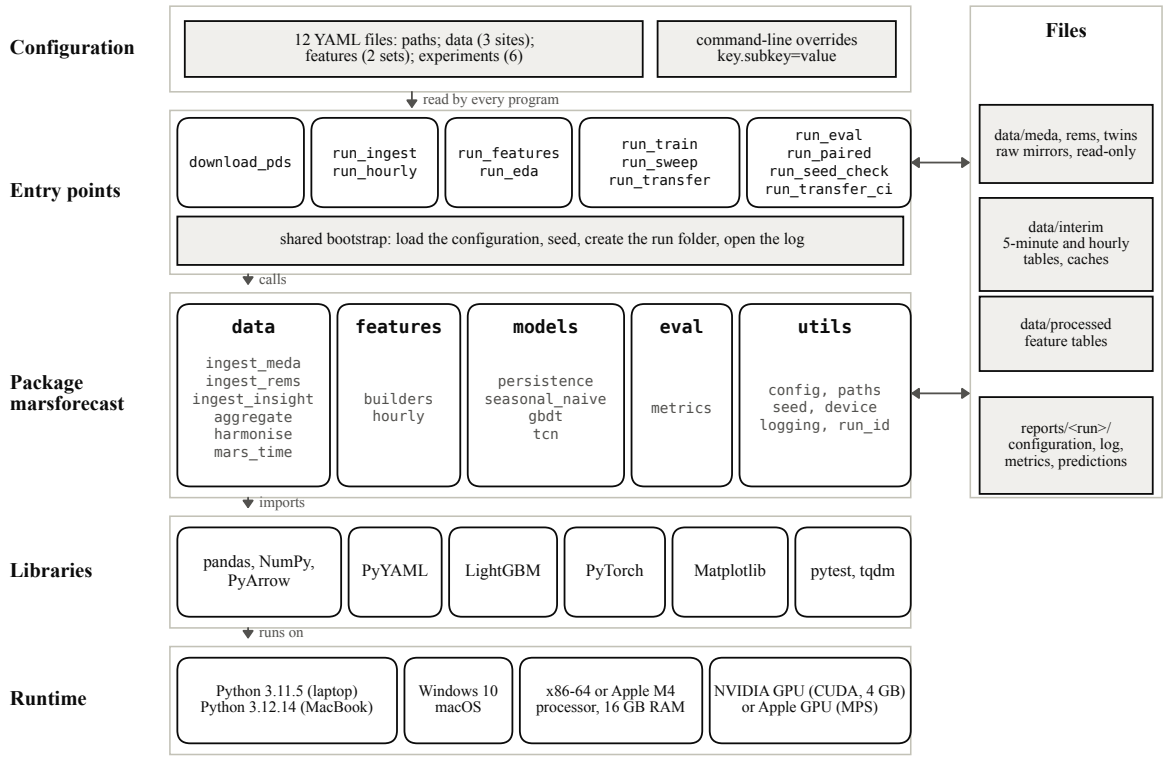
Configuration. Twelve files in the YAML format, a human-readable data serialization language[6], hold every parameter: a path map, one data file per station (source folders and file patterns, sol range, excluded sensor periods, grid length, minimum samples per hour, output paths), one file per feature set (channels, lags, windows, split ratios, output folder) and one file per experiment (data and feature files, model class and hyperparameters, horizons, seed). A file may extend another, in which case its keys override the parent's key by key, and the command line may override any key with a `key.subkey=value` argument, which is applied last. The experiment file of RQ2, for example, extends the GBDT file of RQ1 and changes two lines, the name and the feature set, so that the two runs differ in nothing else. The five seeds of the seed check (Chapter 7) were set with one command-line override each. Each run folder holds the configuration as resolved, so a run can be repeated from a single file.

Entry points. Thirteen programs form the user interface of the system. Eight of them start through a shared bootstrap that parses the configuration path and the overrides, loads and merges the configuration, seeds the random number generators, creates the run folder with the resolved configuration and the code version, and opens a log that is written both to the console and to a file. The download program takes an archive address, a file-name pattern and a destination instead, and the remaining four, which compare finished runs (the metrics table, the paired comparison, the seed check and the transfer intervals), take run folders and write their results next to the runs they read. Each program is thin: it reads its inputs, calls functions of the package in a fixed order and writes its outputs.

Package. The package `marsforecast` has five subpackages (Table 5.2). The data layer reads the three archives and produces the 5-minute and hourly tables; the feature layer turns the tables into model inputs and targets; the model layer holds the four model classes; the evaluation layer computes the metrics; and the utilities load configurations, resolve paths, seed, log and select the computing device. The two learned models expose a training function that returns a fitted state and a prediction function that maps a frame of issue times to forecasts; the reference forecasts expose a prediction function, and the typical day also a fitting function. The training program

[6] YAML Language Development Team. *YAML Ain't Markup Language (YAML) version 1.2*. Revision 1.2.2. Oct. 1, 2021. URL: `https://yaml.org/spec/1.2.2/` (accessed 2026-10-28).

Figure 5.1: Layers of the software system and the files it exchanges



Source: own elaboration based on the project code.

calls each class in its own branch and then scores all four with one code path on the same rows, which is what requirement R2 demands. The routing from a model name to its module is a conditional in the training program rather than a registry; the code acknowledges this as a point for future cleanup. The two model libraries, LightGBM and PyTorch, are imported only by the run that uses them and never together: on macOS each brings its own OpenMP runtime, and a process that loads both crashes as soon as the second runs threads. A test enforces this separation.

Libraries and runtime. The package builds on pandas and NumPy for tables and arrays[7], PyArrow for the Parquet files, PyYAML for the configuration, LightGBM for the GBDT[8], PyTorch for the TCN[9], Matplotlib for the figures and pytest for the tests; apart from the progress bars of tqdm and the Markdown tables of tabulate, nothing else is required at run time. The layering is also visible in the size of the code: the model layer is the largest part of the package

[7]W. McKinney. "Data Structures for Statistical Computing in Python". In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61; C. R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (2020), pp. 357–362.

[8]G. Ke et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, pp. 3146–3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html (accessed 2026-10-07).

[9]A. Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (accessed 2026-10-07).

Table 5.2: Modules of the package and their responsibilities

Subpackage	Module	Responsibility	Lines
`data`	`ingest_meda`	reads the MEDA files sol by sol, replaces the invalid-value code, parses local time	104
	`aggregate`	5-minute means on the clock grid, wind vector components, merging of intervals split between files, join of the sensors	120
	`ingest_rems`	reads MODRDR tables by column position, applies the confidence bytes and the off-scale boom rule	64
	`ingest_insight`	reads the TWINS and PS files, applies the operational flags	68
	`harmonise`	hourly grid of LMST hours, sample-weighted merging of files, wind components	101
	`mars_time`	LMST parsing, spacecraft clock to UTC, solar longitude (Mars24)	56
`features`	`builders`	missingness flags, calendar encodings, short-gap fill, chronological split, flux cap, pressure baseline, lags and windows, targets, input selection	264
	`hourly`	complete hourly grid, relative features and change targets for the transfer experiments	77
`models`	`persistence`	last observed value	12
	`seasonal_naive`	time-of-sol climatology at the target time	43
	`gbdt`	LightGBM training with early stopping, prediction	68
	`tcn`	network, sequence builder and scaler, training loop, prediction	432
`eval`	`metrics`	RMSE, MAE, skill score, number of scored rows	37
`utils`	`config`	YAML loading, inheritance, command-line overrides	44
	`paths`	project root and path map	29
	`run_id`	run identifier, resolved configuration, code version	37
	`seed`	seeding of Python, NumPy and PyTorch	37
	`logging`	console and file log	28
	`device`	selection of CUDA, Apple GPU or processor	23

The six package-initialization files add 93 lines. MODRDR, TWINS, PS: the REMS and InSight products of Chapter 3; LMST: local mean solar time; RMSE, MAE: root mean square and mean absolute error.

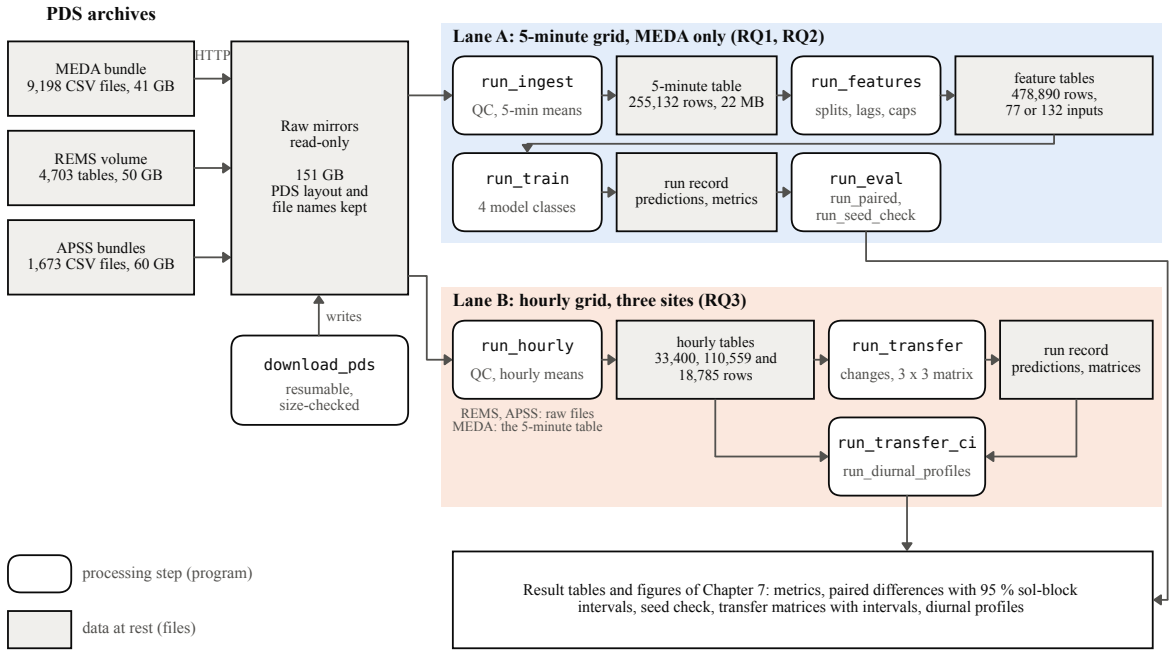
Source: own elaboration based on the project code.

because the TCN carries its own data loader, scaler and training loop, whereas the GBDT delegates these to its library. One weakness follows from the thinness of the package: the orchestration of a training run, that is, the construction of the targets at each horizon, the choice of the split and the routing to a model, lives in the training program, so the tuning program repeats part of it. Moving this logic into the package would remove the duplication.

5.4 Data architecture

Figure 5.2 traces the data from the archives to the result tables. The storage is divided into four zones, which correspond to the folders of Chapter 3: the raw mirrors, the interim tables, the processed feature tables and the run records. The zones differ in who may write them and in what they contain. The raw mirrors are written only by the download program, keep the directory layout and the file names of the archives, and are treated as read-only by every other program, a

Figure 5.2: Data flow from the archives to the result tables



Source: own elaboration based on the project code and the tables of Chapters 3 and 4.

practice recommended for scientific data[10]. The interim zone holds the per-sensor caches and the 5-minute table of MEDA and the hourly table of each station; the processed zone holds one feature table per feature set; and the run records hold what each run produced. Everything outside the raw zone can be rebuilt from it with the programs of Table 5.3.

The data flow has two lanes, one per time grid. Lane A serves the single-site experiments: the MEDA files are aggregated to the 5-minute table, the feature program adds the split labels, flags, encodings, lags and windows and the training-period statistics, and the training program fits one model per target and horizon and writes its forecasts for every test issue time. Lane B serves the transfer experiments: the hourly program builds one table per station, from the raw files for REMS and InSight and from the 5-minute table for MEDA, and the transfer program trains and scores the GBDT on every combination of training and test site. The comparison programs of both lanes read only run records. Every stage reduces the data: 151 GB of archive files become a 22 MB 5-minute table and 8 MB of hourly tables, the feature tables grow again to 121 and 221 MB because they hold 98 and 148 columns on the complete grid of 478,890 intervals, and a training run leaves 3 to 5 MB of forecasts.

All tables are stored as Apache Parquet files, an open, column-oriented format with typed columns, explicit null values and compression[11], written and read with pandas and PyArrow. Metrics and most summaries are JSON files, the human-readable tables are CSV, Markdown

[10] Cf. Wilson et al., *op. cit.*

[11] Apache Software Foundation. *Overview*. Apache Parquet documentation. URL: `https://parquet.apache.org/docs/overview/` (accessed 2026-10-28).

Table 5.3: Pipeline stages, programs, their inputs and outputs and typical run times

Stage	Program	Reads	Writes	Run time (machine)
Download	`download_pds`	archive directory listings or an index file, over HTTP	raw mirrors	hours to days per archive, at about 0.5 MB s^{-1} per connection
Ingest	`run_ingest`	MEDA files or the per-sensor caches	5-minute table; summary and missing fractions	75 min from the files, seconds from the caches
Hourly tables	`run_hourly`	5-minute table (MEDA) or raw files (REMS, InSight)	hourly table of one station; summary	2 min for REMS and 4 min for InSight with 8 processes (MacBook)
Features	`run_features`	5-minute table	feature table with split labels; summary	about 2 s per feature set
Training and scoring	`run_train`	feature table	forecasts and metrics of the test period; TCN learning curves	references 1 s; GBDT 37–40 s (MacBook); TCN 31 min (laptop GPU)
TCN tuning	`run_sweep`	feature table	validation error of each setting	19 min (laptop GPU)
Transfer	`run_transfer`	hourly tables	forecasts, long table and matrices of the 3×3 experiment	about 3.5 min (MacBook)
Analysis	`run_eval`, `run_paired`, `run_seed_check`, `run_transfer_ci`, `run_diurnal_profiles`, `run_eda`	run records; hourly tables (diurnal profiles); 5-minute table (EDA)	metrics table in LaTeX; paired differences, seed check and transfer gaps with bootstrap intervals; diurnal profiles; exploratory statistics	seconds to a few minutes

EDA: exploratory data analysis. Run times are those of the runs reported in Chapter 7.
Source: own elaboration based on the run logs.

or LaTeX, and the configuration is YAML. Each table has a key that the later stages join on: the 5-minute table and the feature tables are keyed by the index of the 5-minute interval of the spacecraft clock; the hourly tables by the station and the Mars-hour index; and the forecasts by the target, the horizon and the interval index. Because the paired comparison and the seed check join forecasts on these keys, two models are compared only on issue times that both scored, which is how requirement R2 is enforced after the fact. Both grids are completed before any lag is formed, with an empty row for every missing interval or hour: the feature program stores the feature table on all 478,890 intervals, and the transfer program completes the hourly grid in memory, so that a lag of one hour means one hour and not one row (Chapter 4).

Two further properties of the data architecture follow from the requirements. First, the split labels are columns of the feature table, written once by the feature program, together with every training-period statistic; a model therefore cannot see a test row in training unless the feature program put it there, and all models read the same labels (R3). Second, the system stores forecasts, not models. The folder reserved for serialized models is empty: a training run leaves

its forecasts for every test issue time, from which all metrics, intervals and figures of Chapter 7 can be recomputed without retraining, but a forecast for a new issue time requires training the model again. This is adequate for a study and inadequate for a service (Section 5.8).

5.5 Integration and network

The parts of the system are integrated through files with fixed schemas, not through calls between running processes. A stage reads only the configuration and the outputs of earlier stages and writes only into its own output folder, so every interface is a file whose columns are documented in the codebook of Chapter 3. This makes each stage independently rerunnable and testable, and it makes the interfaces inspectable with any tool that reads Parquet, JSON or YAML. Within the training program, the four model classes share one function interface and one scoring path. Between the project and the thesis, the integration is also by files: the metrics program emits a LaTeX table, the paired comparison emits LaTeX and Markdown and the other comparisons Markdown and CSV, the figures of Chapters 3 and 4 are drawn by scripts stored with the chapters that read only named tables and run records, and the diagrams of this chapter by a script without data input. Between the two machines, the integration was a snapshot folder with the code and the feature table, whose checksum was recorded (Section 5.8).

The network is used by one program only. The download program crawls the directory listings of the archive node, or takes a list of file paths built from the index table of a volume, and fetches the matching files over HTTP with up to eight connections, writing each file to a temporary name first, comparing its size with the size announced by the server and keeping only complete files, so that an interrupted download can be resumed by rerunning the same command. The program identifies itself to the server with a user-agent string that names the project. The archive node allowed about 0.5 MB s^{-1} per connection, so the 151 GB took several sessions over days. After the download, no component opens a network connection: ingest, training, scoring and analysis run offline, the system listens on no port, and it uses no remote or cloud resource. Interoperability rests on open formats and standard libraries: the CSV and fixed-width inputs are those of the archives, and the Parquet, JSON, CSV and YAML outputs can be read from other languages and tools.

5.6 Security, integrity and licensing

The security requirements of the system are modest, because its assets are public data, the author's code and the results. The archives are in the public domain[12] and contain no personal data (Chapter 3), the download needs no authentication, and the configuration files hold no credentials or secrets. There is no multi-user access to protect: both machines are personal computers used by one person, and the code carries the MIT license. The threats that do matter are to integrity:

[12] PDS Geosciences Node. *Frequently Asked Questions*. URL: `https://pds-geosciences.wustl.edu/faq.html` (accessed 2026-10-18).

corrupted or truncated inputs, silently wrong results, drifting dependencies and the loss of the link between a result and the code that produced it.

Input integrity is protected at three points. The download verifies file sizes and never leaves a partial file under its final name; the raw mirrors keep the archive names, so any file can be checked against its source again; the ingest programs convert every missing-value code and apply the quality-control rules of Chapter 3 in code, with the thresholds as named constants; and the mirrors hold one product version per sol (Chapter 3). Result integrity rests on the measures of Chapter 4 and Section 5.4: statistics fitted on the training period only, split labels stored in the data, scoring restricted to measured targets, and comparisons joined on identical rows. When the TCN was trained on the laptop, the feature table was transferred with its SHA-256 checksum and the results were checked on the MacBook: the same 236,127 test rows and target values as the GBDT run, finite forecasts, and metrics that equal those recomputed from the stored forecasts. Dependency integrity rests on the pinned versions, which also limit the attack surface of the software supply chain to a known set of packages.

Determinism is handled explicitly and has known limits. Every program started through the bootstrap seeds Python, NumPy and, where loaded, PyTorch, and the comparison programs take the seed of their resampling as an argument. The GBDT runs with LightGBM's deterministic mode and a fixed row-wise histogram construction, and its runs reproduce exactly across the two machines. The seed of the configuration, however, did not reach LightGBM in the single runs, whose row and feature sampling used the library's default seeds[13]; the seed check of Chapter 7, which sets the LightGBM seed directly, was the response. The TCN relaxes determinism on purpose: deterministic convolution algorithms slowed an epoch from about twenty seconds to twenty minutes on the laptop GPU, so the algorithm selection of cuDNN is left free, and PyTorch does not guarantee identical results across runs or platforms in any case[14]. The observed run-to-run variation of the TCN, about 6 % of the validation error, is reported in Chapter 7 and treated as noise there.

The most serious gap is version control. The project folder was not a Git repository during the work, so every run identifier ends in `nogit` instead of a commit hash, and a result cannot be tied to a code revision by the run record alone. The gap was mitigated, not closed: since October 2026 every change that alters results is recorded in a change log with the evidence for the change and, for code changes, a patch file, and the chapters name the run identifiers behind their numbers. A repository with one commit per change, as the project had planned and as good practice recommends[15], remains the correct solution and is proposed in Chapter 8.

[13] Cf. Microsoft Corporation. *Parameters*. LightGBM 4.3.0 documentation. 2024. URL: `https://lightgbm.readthedocs.io/en/v4.3.0/Parameters.html` (accessed 2026-10-28).

[14] Cf. PyTorch Contributors. *Reproducibility*. PyTorch 2.2 documentation. 2024. URL: `https://docs.pytorch.org/docs/2.2/notes/randomness.html` (accessed 2026-10-28).

[15] Cf. Wilson et al., *op. cit.*

5.7 Observability and testing

Every run is observable through its folder under `reports/`, named by date, time, configuration and code version (Table 5.4). The folder is created before any work starts, with the resolved configuration, the code version and an empty metrics file; the metrics are written only when the run has produced forecasts, so an unfinished run is recognizable by its empty metrics file, and the metrics program skips such folders when it collects the latest run of each model. The log carries time stamps, the sizes of the tables read, the number of inputs selected, and one line per fitted model with its error and the number of scored rows; during training, progress bars report boosting rounds or epochs on the console. The TCN additionally stores its learning curve per model, and its training stops with an error if an epoch ends with a non-finite loss, so that a diverged network cannot end silently at an early best epoch. As of 28 October 2026 the reports folder holds 77 run folders and 0.2 GB. Superseded runs are kept, and the chapters state which runs they use.

Table 5.4: Contents of a run record

File	Content	Written by
`config.yaml`	the configuration after inheritance and overrides	the bootstrap, at the start
`git_sha.txt`	the code version, or `nogit`	the bootstrap, at the start
`log.txt`	the time-stamped log of the run	the bootstrap and the program
`metrics.json`	model, feature set, targets, horizons; RMSE, MAE and row count per target and horizon	training programs, at the end
`predictions.parquet`	one forecast per test issue time, target and horizon, with the measured value	training and transfer programs
`history/*.json`	training and validation error per epoch	TCN runs
summaries	ingest summary and missing fractions; hourly summary; feature summary with the flux cap and the split sizes; sweep table; transfer tables, long table and intervals	the respective program

Source: own elaboration based on the project code.

Correctness is checked by 46 automated tests in ten files, written with pytest[16]; all pass on the MacBook as of 28 October 2026. They cover the configuration loader (inheritance and overrides), the MEDA ingest (vector averaging of wind, intervals split between files), the REMS quality control, the hourly harmonization and Mars time (the solar longitude is checked against known dates), the feature construction (short-gap fill, measured-target masks, the exclusion of secondary channels from the primary feature set, causality of the relative features), the typical-day forecast, the TCN (training statistics from observed values only, missing inputs at the training mean, finite errors on the default device, the optimizer, device selection, abort on divergence), the transfer

[16]H. Krekel et al. *pytest: helps you write better programs*. pytest 8.2 documentation. 2024. URL: `https://docs.pytest.org/en/8.2.x/` (accessed 2026-10-28).

splits, the decision rule of the seed check, and the separation of the two model libraries, which is tested in subprocesses so that a crash fails the test rather than the test session. The tests for the preprocessing defects found in the review of Chapter 4 were written before the fixes and failed on the old code. Two limitations remain: the tests are run by hand, as there is no continuous integration, and they test units rather than a complete run on the real data, whose checks are the comparisons between runs described above.

5.8 Deployment

5.8.1 Deployment of the experiments

The system is deployed by installing the pinned environment and the package in editable mode, placing the archives under the data folder and running the programs in the order of Table 5.3. The complete single-site experiment of RQ1 is, for one model class,

```
python scripts/run_ingest.py   --config config/data/meda.yaml
python scripts/run_features.py --config config/features/primary_only.yaml
python scripts/run_train.py    --config config/experiments/meda_gbdt.yaml
python scripts/run_eval.py     --auto --latex
```

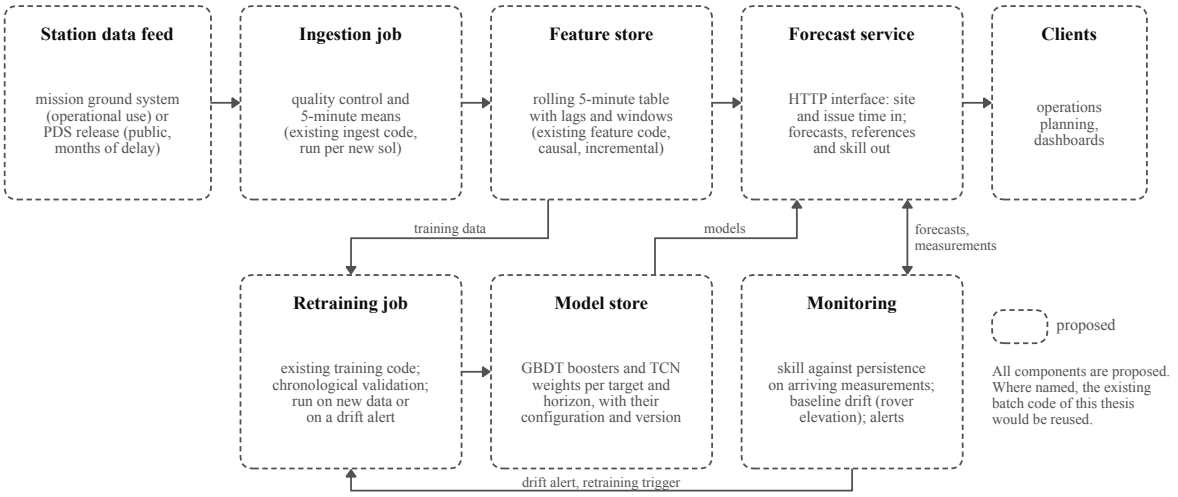
and the other model classes, the second feature set and the transfer experiment differ only in the configuration file named. Two deployments were used. On the laptop the environment was a pip virtual environment with the CUDA build of PyTorch; on the MacBook, a pip virtual environment with the arm64 build, with the OpenMP runtime that LightGBM needs installed through Homebrew. For the final TCN run, the MacBook's code and its rebuilt feature table were copied to the laptop as a snapshot folder with its own instructions and check scripts. The checksum of the table was recorded, and the record of the laptop runs lists the checks made there: both runs present, finite errors, test rows identical to the MacBook's, learning curves stored, CUDA and the AdamW optimizer used, and the tuning winner applied in the final run. The results were then copied back and verified again (Section 5.6). The handoff shows the cost of a two-machine deployment without version control: nothing but these checks tied the snapshot to the code on the MacBook.

5.8.2 Outline of a forecasting service

Chapter 1 described the software as a research pipeline and promised an outline of its deployment as a forecasting service. Such a service is not implemented; Figure 5.3 shows what it would consist of, which parts of the present code it could reuse and what would have to be added. Its constraint is the data feed. The archives are published months after the measurements: the MEDA release used here was published in December 2025 and ends with a sol of September 2025[17].

[17] J. A. Rodriguez-Manfredi et al. *The Mars Environmental Dynamics Analyzer, MEDA*. Data bundles of the Mars 2020 mission. NASA Planetary Data System, Atmospheres Node. 2021. URL: https://pds-atmospheres.nmsu.edu/data_and_services/atmospheres_data/PERSEVERANCE/meda.html (accessed 2026-10-07). DOI: 10.17189/1522849, release notes.

Figure 5.3: Outline of a deployment as a forecasting service



Source: own elaboration.

A service fed from the public archives could therefore only replay the past, and an operational service would have to receive the station data from the ground data system of a mission, where the measurements arrive within sols. The outline assumes such a feed and makes no claim about access to it.

Given the feed, the ingestion job would run the existing quality control and 5-minute aggregation on each new sol file, which the ingest code already processes one file at a time. The feature store would keep a rolling 5-minute table with the features of Chapter 4. These features are causal and bounded in time: the longest lag and window span 24 hours and the TCN reads 24 hours of inputs, so a buffer of the last two sols per station suffices, while the flux cap and the seasonal pressure baseline are fixed parameters from training. The model store is the largest addition, because the present system does not persist models. It would hold, for each target and horizon, the GBDT booster and the TCN weights together with the resolved configuration, the training period and a version. The forecast service would expose one operation, forecasts for a station and an issue time, and return the twelve forecasts with the two reference forecasts and the skill of the model over the training period, so that a user sees the forecast and its expected worth together. Inference is cheap: the twelve GBDT models forecast an issue time in a fraction of a second, and the 222,529-parameter TCN needs no GPU for inference. A retraining job would rerun the existing training code with chronological validation when a release arrives or when monitoring reports drift; the 36 Pa shift of pressure with the rover's elevation found in Chapter 4 is the kind of change that such monitoring would have to detect. Monitoring would compute the skill against persistence as the measured values arrive, which the present metrics code already does for stored forecasts.

The resources would be small. The GBDT models retrain in under a minute, the TCN in about half an hour on a 4 GB GPU, and the derived data occupy under a gigabyte, so one small virtual

machine would host the service, with a GPU only for TCN retraining. The pinned environment would be packaged as a container image, which fixes the dependencies for consistent deployment across machines[18], and the service would need authentication, alerts and a record of the model version behind each forecast. The surveys of deployed machine-learning systems place most of the difficulty of such a transition in data management, monitoring and the maintenance of models rather than in the models themselves[19], which agrees with this outline: the models of this thesis are the smallest part of a service built around them.

5.9 Assessment against design considerations

Table 5.5 assesses the architecture against the design considerations commonly applied to information systems; each entry names what the system does and what it lacks.

Table 5.5: Assessment of the architecture against common design considerations

Consideration	Measures taken; limitations
Scalability	one sol of one sensor in memory at a time, 8 processes for the largest archives; one machine, and a new model class needs code
Reliability	resumable downloads, a failed file does not stop a site, unfinished runs are marked and skipped, divergence aborts training; no retries beyond the download, no backup policy
Performance	caches, complete grids and pre-built tensors; a full rerun takes minutes, except the TCN (31 min on the GPU) and the first ingest (75 min)
Security and integrity	public data, no secrets, no network after the download, size checks, read-only mirrors, pinned versions, checksummed handoff; no version control during the work
Maintainability	layered package, thin programs, 46 tests, configuration-driven experiments, change log; routing by conditional, duplicated orchestration, unused dependencies, one unused key
Interoperability	open formats and standard libraries; no limitation for the present use
Cost	consumer hardware and free software; under two hours of compute for the final runs
Observability	run records with configuration, log, metrics, forecasts and learning curves; no alerts
Extensibility	stations, feature sets, horizons and hyperparameters by configuration, models by one module and one branch; no model persistence, no service

Source: own elaboration.

The system therefore does what a study needs: every number of Chapter 7 is reproducible from its run record, and the models, feature sets and sites are varied by configuration alone. Its debts are those of a single-author research pipeline, above all the missing version control, the orchestration logic that sits in a program rather than in the package, and the absence of model persistence. None of them affects the results, and all of them would have to be settled before the system served anyone but its author.

[18] Cf. D. Merkel. “Docker: Lightweight Linux Containers for Consistent Development and Deployment”. In: *Linux Journal* 2014.239, 2 (2014).

[19] Cf. A. Paleyes, R.-G. Urma, and N. D. Lawrence. “Challenges in Deploying Machine Learning: A Survey of Case Studies”. In: *ACM Computing Surveys* 55.6 (2022), pp. 1–29.

Bibliography

Amershi, S. et al. “Software Engineering for Machine Learning: A Case Study”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 291–300. DOI: 10.1109/ICSE-SEIP.2019.00042.

Apache Software Foundation. *Overview*. Apache Parquet documentation. URL: https://parquet.apache.org/docs/overview/ (accessed 2026-10-28).

Bass, L., Clements, P., and Kazman, R. *Software Architecture in Practice*. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2012.

Harris, C. R. et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

Ke, G. et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, pp. 3146–3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html (accessed 2026-10-07).

Krekel, H. et al. *pytest: helps you write better programs*. pytest 8.2 documentation. 2024. URL: https://docs.pytest.org/en/8.2.x/ (accessed 2026-10-28).

McKinney, W. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

Merkel, D. “Docker: Lightweight Linux Containers for Consistent Development and Deployment”. In: *Linux Journal* 2014.239, 2 (2014).

Microsoft Corporation. *Parameters*. LightGBM 4.3.0 documentation. 2024. URL: https://lightgbm.readthedocs.io/en/v4.3.0/Parameters.html (accessed 2026-10-28).

Paleyes, A., Urma, R.-G., and Lawrence, N. D. “Challenges in Deploying Machine Learning: A Survey of Case Studies”. In: *ACM Computing Surveys* 55.6 (2022), pp. 1–29. DOI: 10.1145/3533378.

Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (accessed 2026-10-07).

PDS Geosciences Node. *Frequently Asked Questions*. URL: https://pds-geosciences.wustl.edu/faq.html (accessed 2026-10-18).

PyTorch Contributors. *Reproducibility*. PyTorch 2.2 documentation. 2024. URL: https://docs.pytorch.org/docs/2.2/notes/randomness.html (accessed 2026-10-28).

Rodriguez-Manfredi, J. A. et al. *The Mars Environmental Dynamics Analyzer, MEDA*. Data bundles of the Mars 2020 mission. NASA Planetary Data System, Atmospheres Node. 2021. URL: https://pds-atmospheres.nmsu.edu/data_and_services/atmospheres_data/PERSEVERANCE/meda.html (accessed 2026-10-07). DOI: 10.17189/1522849.

Sandve, G. K. et al. “Ten Simple Rules for Reproducible Computational Research”. In: *PLoS Computational Biology* 9.10, e1003285 (2013). DOI: 10.1371/journal.pcbi.1003285.

Sculley, D. et al. “Hidden Technical Debt in Machine Learning Systems”. In: *Advances in Neural Information Processing Systems*. Vol. 28. Curran Associates, Inc., 2015. URL: https://papers.nips.cc/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html (accessed 2026-10-28).

Wilson, G. et al. “Good enough practices in scientific computing”. In: *PLoS Computational Biology* 13.6, e1005510 (2017). DOI: 10.1371/journal.pcbi.1005510.

YAML Language Development Team. *YAML Ain’t Markup Language (YAML) version 1.2*. Revision 1.2.2. Oct. 1, 2021. URL: https://yaml.org/spec/1.2.2/ (accessed 2026-10-28).

Zaharia, M. et al. “Accelerating the Machine Learning Lifecycle with MLflow”. In: *IEEE Data Engineering Bulletin* 41.4 (2018), pp. 39–45.